Lecture 37 Practice Problems: Complete Step-by-Step Solutions

---

Solution 1: Forward Pass Output Computation

1. **Pre-activation linear combination** z:

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$z = (0.4)(2.0) + (-0.6)(-1.0) + (0.2)(3.0) + (-0.5)$$

$$z = 0.8 + 0.6 + 0.6 - 0.5 = 1.5$$

2. **Sigmoid activation** $\hat{y}$:

$$\hat{y} = \sigma(1.5) = \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.22313} \approx 0.8176$$

Solution 2: Outlier Sensitivity in Regression Losses

Given $y = 10.0, \hat{y} = 60.0 \implies$ error $e = y - \hat{y} = 10.0 - 60.0 = -50.0$.

1. **MSE Loss**:

$$\mathcal{L}_{MSE} = \frac{1}{2}e^2 = \frac{1}{2}(-50.0)^2 = \frac{1}{2}(2500) = 1250.0$$

2. **MAE Loss**:

$$\mathcal{L}_{MAE} = |e| = |-50.0| = 50.0$$

3. **Huber Loss ($\delta = 2.0$)**: Since $|e| = 50.0 > \delta = 2.0$, we evaluate the linear branch:

$$\mathcal{L}_{\text{Huber}} = \delta|e| - \frac{1}{2}\delta^2 = (2.0)(50.0) - \frac{1}{2}(2.0)^2 = 100.0 - 2.0 = 98.0$$

- **Analysis**: MSE inflated the loss by $25\times$ compared to MAE and $12.7\times$ compared to Huber Loss. Huber loss and MAE prevented gradient explosion because their derivative

with respect to large errors is bounded by δ and 1.0 respectively, whereas MSE's derivative scales linearly with error ($|-50| = 50$).

Solution 3: Binary Cross-Entropy Penalty Analysis

1. **Sample A ($y_A = 1, \hat{y}_A = 0.95$)**:

$$\mathcal{L}_A = -\log(0.95) \approx 0.0513$$

2. **Sample B ($y_B = 1, \hat{y}_B = 0.05$)**:

$$\mathcal{L}_B = -\log(0.05) \approx 2.9957$$

3. **Ratio of Penalties**:

$$\frac{\mathcal{L}_B}{\mathcal{L}_A} = \frac{2.9957}{0.0513} \approx 58.4\times$$

- **Consequence**: The model receives nearly $60\times$ larger penalty for confident incorrect prediction, forcing backpropagation to adjust parameters drastically.

Solution 4: Softmax Probability Distribution & Categorical Cross-Entropy

Given $z = [2.0, 1.0, 0.1]$ and $y = [1, 0, 0]$:

1. **Softmax Computation**:

$$e^{z_1} = e^{2.0} \approx 7.3891, \quad e^{z_2} = e^{1.0} \approx 2.7183, \quad e^{z_3} = e^{0.1} \approx 1.1052$$

$$\sum_{j=1}^{3} e^{z_j} = 7.3891 + 2.7183 + 1.1052 = 11.2126$$

$$\hat{y}_1 = \frac{7.3891}{11.2126} \approx 0.6590, \quad \hat{y}_2 = \frac{2.7183}{11.2126} \approx 0.2424, \quad \hat{y}_3 = \frac{1.1052}{11.2126}$$

Normalized probability vector: $\hat{y} = [0.6590, 0.2424, 0.0986]$.

2. **Categorical Cross-Entropy**:

$$\mathcal{L} = -\sum_{c=1}^{3} y_c \log(\hat{y}_c) = -[1 \cdot \log(0.6590) + 0 + 0] = -(-0.4170) \approx$$

Solution 5: Analytic Derivation of Softmax Derivative

Let $\hat{y}_i = \frac{e^{z_i}}{S}$ where $S = \sum_k e^{z_k}$. Using quotient rule $\frac{\partial}{\partial z_j}\left(\frac{u}{v}\right) = \frac{u'v - uv'}{v^2}$:

- Note that $\frac{\partial S}{\partial z_j} = e^{z_j}$.
- **Case 1:** $i = j$:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} = \frac{e^{z_i}}{S} \cdot \frac{S - e^{z_i}}{S} = \hat{y}_i(1 - \hat{y}_i)$$

- **Case 2:** $i \neq j$:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \frac{0 \cdot S - e^{z_i} e^{z_j}}{S^2} = -\frac{e^{z_i}}{S} \cdot \frac{e^{z_j}}{S} = -\hat{y}_i \hat{y}_j$$

Combining both cases using Kronecker delta δ_{ij}:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j) \quad \blacksquare$$

Solution 6: Single-Step Gradient Descent Update

1. Prediction: $\hat{y} = (2.5)(1.5) + 0.8 = 3.75 + 0.8 = 4.55$.
2. Error: $e = y - \hat{y} = 4.0 - 4.55 = -0.55$.
3. Gradients of $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$:

$$\frac{\partial \mathcal{L}}{\partial w} = -(y - \hat{y}) \cdot x = -(-0.55)(1.5) = 0.825$$

$$\frac{\partial \mathcal{L}}{\partial b} = -(y - \hat{y}) \cdot 1 = -(-0.55) = 0.55$$

4. Parameter Updates ($\eta = 0.1$):

$$w_{\text{new}} = 2.5 - (0.1)(0.825) = 2.5 - 0.0825 = 2.4175$$

$$b_{\text{new}} = 0.8 - (0.1)(0.55) = 0.8 - 0.055 = 0.745$$

Solution 7: Backpropagation via Chain Rule

1. Forward values:
   - $z = (1.0)(3.0) + (-1.0) = 2.0$.
   - Since $z = 2.0 > 0$, $a = \text{ReLU}(2.0) = 2.0$.
   - Loss: $\mathcal{L} = \frac{1}{2}(5.0 - 2.0)^2 = \frac{1}{2}(9.0) = 4.5$.
2. Local derivatives:
   - $\frac{\partial \mathcal{L}}{\partial a} = -(y - a) = -(5.0 - 2.0) = -3.0$.
   - $\frac{\partial a}{\partial z} = 1.0$ (since $z > 0$).
   - $\frac{\partial z}{\partial w} = x = 3.0$.
3. Composite gradient:

$$\frac{\partial \mathcal{L}}{\partial w} = (-3.0) \cdot (1.0) \cdot (3.0) = -9.0$$

Solution 8: Proof of Sigmoid Gradient Upper Bound

1. Given $\sigma(z) = \frac{1}{1+e^{-z}}$, derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
2. Let $u = \sigma(z) \in (0, 1)$. We seek the maximum of $g(u) = u(1 - u) = u - u^2$.
3. Differentiating with respect to u: $g'(u) = 1 - 2u = 0 \implies u = 0.5$.
4. Second derivative: $g''(u) = -2 < 0 \implies$ strict global maximum.
5. Evaluating at $u = 0.5$: $g(0.5) = (0.5)(0.5) = 0.25$. Since $\sigma(z) = 0.5 \iff z = 0$, maximum occurs at $z = 0$.
6. Across an 8-layer network:

$$\prod_{l=1}^{8} \sigma'(z_l) \leq (0.25)^8 = \frac{1}{65,536} \approx 1.525 \times 10^{-5}$$

Gradients decay by over 99.998%, proving vanishing gradients. ■

Solution 9: The Dying ReLU Pathology

1. Pre-activation $z = 0.5x_1 - 0.2x_2 - 2.0$. Maximum possible value occurs when $x_1 = 1, x_2 = 0$:

$$z_{\max} = 0.5(1) - 0.2(0) - 2.0 = 0.5 - 2.0 = -1.5 < 0$$

   Since $z_{\max} = -1.5 < 0$, $z < 0$ for all possible inputs in the dataset.

2. Under standard ReLU, $f'(z) = 0$ for all $z < 0$. Therefore, for every single training sample:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot 0 \cdot x = 0 \implies w_{\text{new}} = w_{\text{old}}$$

   The weights never update, permanently dead.

Solution 10: Leaky ReLU Gradient Flow

1. With Leaky ReLU ($\alpha = 0.01$):

$$f(z) = 0.01z, \quad f'(z) = 0.01$$

2. Gradient computation:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot (0.01) \cdot x \neq 0$$

3. Weight update is non-zero: $\Delta w = -\eta(0.01)\frac{\partial \mathcal{L}}{\partial a}x$. This allows the bias and weights to eventually shift z back towards positive territory, restoring the neuron to life!

Solution 11: Training Epoch, Batch, and Step Calculations

Given $N = 128,000, B = 256, E = 50$:

1. Iterations per epoch:

$$\text{Steps per Epoch} = \frac{128,000}{256} = 500 \text{ iterations}$$

2. Total weight updates:

$$\text{Total Iterations} = 500 \times 50 = 25,000 \text{ iterations}$$

3. Training duration:

$$\text{Total Time} = 25,000 \times 0.120 \text{ s} = 3000 \text{ seconds} = \frac{3000}{3600} \approx 0.833 \text{ h}$$

Solution 12: Gradient Variance Comparison

Let $g_i = \nabla \mathcal{L}_i$ be independent sample gradients with variance $\text{Var}(g_i) = \sigma^2$. The mini-batch gradient is the sample mean $\bar{g}_B = \frac{1}{B}\sum_{i=1}^{B} g_i$. Using standard variance properties for independent random variables:

$$\text{Var}(\bar{g}_B) = \text{Var}\left(\frac{1}{B}\sum_{i=1}^{B} g_i\right) = \frac{1}{B^2}\sum_{i=1}^{B}\text{Var}(g_i) = \frac{1}{B^2}(B\sigma^2) = \frac{\sigma^2}{B}$$

- **Implication**: As batch size B increases, gradient variance drops as $O(1/B)$. Batch size 64 reduces gradient variance by $64\times$ compared to SGD ($B = 1$), dramatically stabilizing convergence.

Solution 13: Momentum Numerical Simulation

Given $\theta_0 = 5.0, v_0 = 0, \beta = 0.9, \eta = 0.05, g_t = 2.0$:

- **Step 1 ($t = 1$)**:

$$v_1 = (0.9)(0) + (1 - 0.9)(2.0) = 0.2$$

$$\theta_1 = \theta_0 - \eta v_1 = 5.0 - (0.05)(0.2) = 5.0 - 0.01 = 4.99$$

- **Step 2 ($t = 2$)**:

$$v_2 = (0.9)(0.2) + (0.1)(2.0) = 0.18 + 0.2 = 0.38$$

$$\theta_2 = \theta_1 - \eta v_2 = 4.99 - (0.05)(0.38) = 4.99 - 0.019 = 4.971$$

- **Step 3 ($t = 3$)**:

$$v_3 = (0.9)(0.38) + (0.1)(2.0) = 0.342 + 0.2 = 0.542$$

$$\theta_3 = \theta_2 - \eta v_3 = 4.971 - (0.05)(0.542) = 4.971 - 0.0271 = 4.9439$$

Solution 14: RMSProp Adaptive Step Scaling

Given $s_0 = 0, \beta = 0.9, \eta = 0.01, \epsilon = 10^{-8}$:

1. **Second Moment** s_1:
   - For Parameter A ($g_A = 10.0$):
   $$s_{1,A} = (0.9)(0) + (0.1)(10.0)^2 = (0.1)(100) = 10.0$$
   - For Parameter B ($g_B = 0.1$):
   $$s_{1,B} = (0.9)(0) + (0.1)(0.1)^2 = (0.1)(0.01) = 0.001$$
2. **Effective Update Steps**:
   - For A:
   $$\Delta\theta_A = -\frac{0.01}{\sqrt{10.0} + 10^{-8}} \times 10.0 = -\frac{0.01}{3.1623} \times 10.0 \approx -0.03$$
   - For B:
   $$\Delta\theta_B = -\frac{0.01}{\sqrt{0.001} + 10^{-8}} \times 0.1 = -\frac{0.01}{0.03162} \times 0.1 \approx -0.03$$
3. **Conclusion**: Despite gradient g_A being $100\times$ larger than g_B (10.0 vs 0.1), RMSProp produced identical normalized step sizes (-0.0316), eliminating ravine oscillations and speeding up plateau traversal!

Solution 15: Full Adam Optimizer Step Calculation

Given $\theta_0 = 1.0, \eta = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, g_1 = 0.5$ at $t = 1$:

1. **Biased Moments**:
   $$m_1 = (0.9)(0) + (1 - 0.9)(0.5) = 0.05$$
   $$v_1 = (0.999)(0) + (1 - 0.999)(0.5)^2 = (0.001)(0.25) = 0.00025$$
2. **Bias-Corrected Moments**:

$$\hat{m}_1 = \frac{m_1}{1-\beta_1^1} = \frac{0.05}{1-0.9} = \frac{0.05}{0.1} = 0.5$$

$$\hat{v}_1 = \frac{v_1}{1-\beta_2^1} = \frac{0.00025}{1-0.999} = \frac{0.00025}{0.001} = 0.25$$

3. **Parameter Update**:

$$\theta_1 = \theta_0 - \frac{\eta}{\sqrt{\hat{v}_1} + \epsilon}\hat{m}_1$$

$$\theta_1 = 1.0 - \frac{0.001}{\sqrt{0.25} + 10^{-8}}(0.5) = 1.0 - \frac{0.001}{0.5}(0.5) = 1.0 - 0.001 =$$

\n